

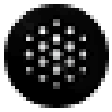

BACKEND SOFTWARE DEVELOPMENT PROJECT (BELLS UNIVERSITY OF TECHNOLOGY-NEW HORIZONS)

PREPARED FOR

200 level Student Team

Bells University of Technology

Course Code: ICT 236

Date: 30-April-2025

PREPARED BY

Ayuba Muhammad(Senior Software Developer)

New Horizons ICT

Dear Student Team,

Re: Enclosed Project Implementation Guidelines

Please find enclosed our detailed project implementation guidelines for your academic team's consideration.

At **Bells University of Technology - New Horizon ICT**, we understand that developing effective, real-world solutions requires a blend of technical skill and teamwork, along with a clear understanding of each team member's role. We encourage each student team to approach this project with dedication and collaboration, as the results will reflect our institution's standard of excellence.

Project Timeline: Each group is given one month from the release date of this project to complete their implementation. This time frame has been carefully designed to allow thorough development while building essential skills in problem-solving, coding, and team-based project management.

Each project must be published on **GitHub** with a detailed README.md and each member assigned as a contributor. Additionally:

- The team leader will be responsible for submitting the final GitHub link.
- Individual members should also fork the repository to showcase contributions.
- For multi-technology integration, we highly recommend using Docker to streamline the process and maintain compatibility across different systems.

Yours Truly,

Instructor.

TEAM

SUMMARY

This document provides detailed guidelines and requirements for the Backend Software Development Project assigned to student teams across the Sciences departments.

Each student group is expected to deliver a complete, original, and deployable software solution using UI/UX Design and selected frameworks. The guidelines within outline the expectations, timelines, and collaborative tools required to ensure a successful project outcome.

Adding Backend Functionality with Blackbox AI Agent

The implementation of a backend system using Blackbox AI agent represents a strategic enhancement to your existing frontend project without requiring architectural changes. By leveraging AI-assisted code generation, your team can rapidly develop a complete server infrastructure with database models, API endpoints, authentication systems, and deployment configurations while maintaining focus on the academic requirements. This approach not only accelerates development but also provides valuable learning opportunities in full-stack architecture, database design, and API development all while ensuring proper documentation and GitHub integration as specified in your project guidelines. The comprehensive implementation guide offers a structured pathway from initial setup through deployment, enabling your team to transform their frontend application into a fully functional solution with minimal friction.

Backend Implementation Strategy

1. Setting Up the Backend Environment

Your team can leverage Blackbox AI to help scaffold a backend that integrates seamlessly with your existing frontend. The process would include:

- Creating a new backend repository that will connect to your existing frontend
- Setting up a Node.js/Express environment for the API
- Implementing a database solution (MongoDB, PostgreSQL, etc.) appropriate for your specific project needs
- Establishing authentication services if required

2. API Development with Blackbox AI

Blackbox AI can help generate the necessary API endpoints that align with your frontend requirements:

1. First, analyze your frontend components to identify all data requirements
2. Use Blackbox AI to generate API endpoint code for each data interaction

3. Implement proper validation, error handling, and security measures
4. Structure the API following RESTful principles or GraphQL depending on your frontend needs

3. Database Design and Implementation

Blackbox AI can assist with:

- Creating database schemas based on your application's data model
- Generating database connection code and ORM/ODM setup
- Implementing proper data relationships and constraints
- Developing data migration strategies if needed

4. Authentication and Authorization

For projects requiring user accounts and access control:

- Implement JWT-based authentication or OAuth 2.0
- Create user registration and login endpoints
- Set up role-based access control for different user types
- Secure API endpoints based on user permissions

5. Deployment and Integration

To connect your existing frontend with the new backend:

- Configure CORS to allow communication between frontend and backend
- Update frontend API calls to connect to the new backend endpoints
- Deploy the backend to a suitable hosting platform (Heroku, Vercel, AWS, etc.)
- Set up CI/CD pipelines for automated testing and deployment

Practical Implementation Steps

<artifact type="text/markdown" id="backend-implementation-guide" title="Backend Implementation Guide Using Blackbox AI"> # Backend Implementation Guide Using Blackbox AI

Overview

This guide explains how to implement a backend system for your frontend project using Blackbox AI assistant. The process is designed to complement your existing frontend without requiring major changes.

Prerequisites

- Completed frontend project (React.js/Next.js)
- GitHub account and Git installed
- Node.js installed locally
- Basic understanding of API concepts

Step 1: Analyze Frontend Requirements

Before starting backend development, document all data requirements from your frontend:

1. **Identify Data Entities**: List all data objects your application needs to store (users, events, posts, etc.)
2. **Map User Flows**: Document user journeys that require data persistence
3. **Define API Requirements**: For each frontend component, list the API endpoints needed

Step 2: Set Up Backend Project

```
# Create a new backend directory

mkdir project-name-backend

cd project-name-backend


# Initialize a new Node.js project

npm init -y


# Install essential dependencies
```

```
npm install express mongoose dotenv cors jsonwebtoken bcrypt


# Create basic directory structure

mkdir -p src/{controllers,models,routes,middleware,config,utils}

touch src/server.js .env .gitignore README.md
```

Step 3: Using Blackbox AI to Generate Backend Code

Database Models

Use Blackbox AI to generate database models based on your frontend requirements:

1. Open your code editor with Blackbox AI extension
2. Create a new file in the models directory (e.g., `src/models/User.js`)
3. Prompt Blackbox AI: "Generate a Mongoose schema for users with fields: name, email, password, role"

Example model that Blackbox might generate:

```
const mongoose = require('mongoose');

const bcrypt = require('bcrypt');



const UserSchema = new mongoose.Schema({

  name: {

    type: String,

    required: [true, 'Name is required'],

    trim: true
```

```
  },
  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    trim: true,
    lowercase: true,
          match:  [/^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
'Please fill a valid email address']
  },
  password: {
    type: String,
    required: [true, 'Password is required'],
    minlength: 8,
    select: false
  },
  role: {
    type: String,
    enum: ['user', 'admin'],
    default: 'user'
  },
  createdAt: {
```

```
    type: Date,
    default: Date.now
  }
});

// Pre-save hook to hash password
UserSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();

  try {
    const salt = await bcrypt.genSalt(10);
    this.password = await bcrypt.hash(this.password, salt);
    next();
  } catch (error) {
    next(error);
  }
});

// Method to compare passwords
UserSchema.methods.comparePassword = async function(candidatePassword) {
  return await bcrypt.compare(candidatePassword, this.password);
```

```
};


module.exports = mongoose.model('User', UserSchema);
```

Repeat this process for all required data models.

API Controllers and Routes

Use Blackbox AI to generate controllers for your API endpoints:

1. Create a new controller file (e.g., `src/controllers/userController.js`)
2. Prompt Blackbox AI: "Generate Express controller functions for user registration, login, and profile management"

Example controller that Blackbox might generate:

```
const User = require('../models/User');
const jwt = require('jsonwebtoken');

// Generate JWT token
const generateToken = (id) => {
  return jwt.sign({ id }, process.env.JWT_SECRET, {
    expiresIn: '30d'
  });
};

// @desc    Register a new user
// @route   POST /api/users
// @access  Public
exports.registerUser = async (req, res) => {
  try {
```

```
    const { name, email, password } = req.body;

    // Check if user exists
    const userExists = await User.findOne({ email });
    if (userExists) {
          return res.status(400).json({ message: 'User already
  exists' });
    }

    // Create new user
    const user = await User.create({
      name,
      email,
      password
    });

    if (user) {
      res.status(201).json({
        _id: user._id,
        name: user.name,
        email: user.email,
        role: user.role,
        token: generateToken(user._id)
      });
    } else {
      res.status(400).json({ message: 'Invalid user data' });
    }
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

// @desc    Authenticate user & get token
// @route   POST /api/users/login
// @access  Public
exports.loginUser = async (req, res) => {
```

```
  try {
    const { email, password } = req.body;

    // Find user by email
    const user = await User.findOne({ email }).select('+password');

    // Check if user exists and password matches
    if (user && (await user.comparePassword(password))) {
      res.json({
        _id: user._id,
        name: user.name,
        email: user.email,
        role: user.role,
        token: generateToken(user._id)
      });
    } else {
      res.status(401).json({ message: 'Invalid email or password' });
    }
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

// @desc    Get user profile
// @route   GET /api/users/profile
// @access  Private
exports.getUserProfile = async (req, res) => {
  try {
    const user = await User.findById(req.user._id);

    if (user) {
      res.json({
        _id: user._id,
        name: user.name,
```

```
        email: user.email,
        role: user.role
      });
    } else {
      res.status(404).json({ message: 'User not found' });
    }
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

// @desc    Update user profile
// @route   PUT /api/users/profile
// @access  Private
exports.updateUserProfile = async (req, res) => {
  try {
    const user = await User.findById(req.user._id);

    if (user) {
      user.name = req.body.name || user.name;
      user.email = req.body.email || user.email;

      if (req.body.password) {
        user.password = req.body.password;
      }

      const updatedUser = await user.save();

      res.json({
        _id: updatedUser._id,
        name: updatedUser.name,
        email: updatedUser.email,
        role: updatedUser.role,
        token: generateToken(updatedUser._id)
      });
    } else {
```

```
      res.status(404).json({ message: 'User not found' });
    }
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

Then create corresponding routes:
// src/routes/userRoutes.js
const express = require('express');
const {
  registerUser,
  loginUser,
  getUserProfile,
  updateUserProfile
} = require('../controllers/userController');
const { protect } = require('../middleware/authMiddleware');

const router = express.Router();

router.post('/', registerUser);
router.post('/login', loginUser);
router.route('/profile')
  .get(protect, getUserProfile)
  .put(protect, updateUserProfile);

module.exports = router;
```

Authentication Middleware

Use Blackbox AI to generate authentication middleware:

```
// src/middleware/authMiddleware.js
const jwt = require('jsonwebtoken');
const User = require('../models/User');
```

```
exports.protect = async (req, res, next) => {
  let token;

  if (
    req.headers.authorization &&
    req.headers.authorization.startsWith('Bearer')
  ) {
    try {
      // Get token from header
      token = req.headers.authorization.split(' ')[1];

      // Verify token
      const decoded = jwt.verify(token, process.env.JWT_SECRET);

      // Get user from token
                                          req.user       =       await
User.findById(decoded.id).select('-password');

      next();
    } catch (error) {
          res.status(401).json({  message:  'Not  authorized,  token
failed' });
    }
  }

  if (!token) {
    res.status(401).json({ message: 'Not authorized, no token' });
  }
};

exports.admin = (req, res, next) => {
  if (req.user && req.user.role === 'admin') {
    next();
  } else {
    res.status(401).json({ message: 'Not authorized as admin' });
  }
```

```
};
```

Server Setup

Use Blackbox AI to generate the main server file:

```
// src/server.js
const express = require('express');
const dotenv = require('dotenv');
const cors = require('cors');
const mongoose = require('mongoose');

// Load env vars
dotenv.config();

// Import routes
const userRoutes = require('./routes/userRoutes');
// Import other routes as needed

// Connect to database
mongoose
  .connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB Connected'))
  .catch(err => console.log('MongoDB connection error:', err));

// Initialize express
const app = express();

// Middleware
app.use(express.json());
app.use(cors());
```

```
// Mount routes
app.use('/api/users', userRoutes);
// Mount other routes as needed

// Basic route
app.get('/', (req, res) => {
  res.send('API is running...');
});

// Error handler
app.use((err, req, res, next) => {
  const statusCode = res.statusCode === 200 ? 500 : res.statusCode;
  res.status(statusCode).json({
    message: err.message,
    stack: process.env.NODE_ENV === 'production' ? null : err.stack
  });
});

// Start server
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Step 4: Environment Configuration

Create a `.env` file:

- NODE_ENV=development
- PORT=5000
- MONGO_URI=mongodb://localhost:27017/your-database-name
- JWT_SECRET=your_jwt_secret_key

Create a `.gitignore` file:

- node_modules
- .env

Step 5: Integrate with Frontend

Update your frontend API calls to point to your new backend:

```
// Example of updating API calls in your frontend
// src/api/index.js

const API_URL = process.env.NEXT_PUBLIC_API_URL ||
'http://localhost:5000/api';

export const loginUser = async (credentials) => {
  const response = await fetch(`${API_URL}/users/login`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(credentials),
  });

  if (!response.ok) {
    throw new Error('Login failed');
  }

  return await response.json();
};

// Add other API functions as needed
```

Step 6: Testing

Use Blackbox AI to generate test cases for your API endpoints:

```
// Example test for user registration using Jest
const request = require('supertest');
const app = require('../src/server');
const User = require('../src/models/User');
const mongoose = require('mongoose');

beforeAll(async () => {
  // Connect to test database
  await mongoose.connect(process.env.MONGO_URI_TEST);
});

afterAll(async () => {
  // Clear database and close connection
  await User.deleteMany({});
  await mongoose.connection.close();
});

describe('User Registration', () => {
  it('should register a new user', async () => {
    const res = await request(app)
      .post('/api/users')
      .send({
        name: 'Test User',
        email: 'test@example.com',
        password: 'password123'
      });
```

```
    expect(res.statusCode).toBe(201);
    expect(res.body).toHaveProperty('token');
    expect(res.body.name).toBe('Test User');
  });

  it('should not register a user with existing email', async () =>
{
    // First create a user
    await User.create({
      name: 'Existing User',
      email: 'existing@example.com',
      password: 'password123'
    });

    // Try to create a user with the same email
    const res = await request(app)
      .post('/api/users')
      .send({
        name: 'New User',
        email: 'existing@example.com',
        password: 'newpassword123'
      });

    expect(res.statusCode).toBe(400);
         expect(res.body).toHaveProperty('message',  'User  already
exists');
  });
});
```

Step 7: Deployment

For deploying your backend:

Create a new repository on GitHub for your backend

Push your code to the repository

Set up deployment on platforms like Heroku, Vercel, or AWS

Example `Procfile` for Heroku:

web: node src/server.js

Step 8: Documentation

Create API documentation using Swagger/OpenAPI with Blackbox AI:

```
// Install swagger packages

// npm install swagger-jsdoc swagger-ui-express


// Add to server.js

const swaggerJsDoc = require('swagger-jsdoc');

const swaggerUi = require('swagger-ui-express');


const swaggerOptions = {

  definition: {

    openapi: '3.0.0',

    info: {

      title: 'Your Project API',

      version: '1.0.0',

      description: 'API documentation for Your Project'

    },
```

```
    servers: [
      {
        url: 'http://localhost:5000',
        description: 'Development server'
      }
    ]
  },
  apis: ['./src/routes/*.js']
};

const swaggerDocs = swaggerJsDoc(swaggerOptions);
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocs));
Then add JSDoc comments to your routes for automatic documentation generation. </artifact>
```

Benefits of Using Blackbox AI for Backend Development

1. **Rapid Development**: Blackbox AI can generate complex backend functionality quickly, allowing your team to focus on customization and integration.

2. **Consistency**: The AI will maintain consistent coding patterns across your backend, making it easier to maintain.

3. **Learning Opportunity**: Team members can learn backend development best practices by reviewing and customizing AI-generated code.

4. **Error Reduction**: AI-generated code is less likely to contain syntax errors or basic logical flaws, though you'll still need to test thoroughly.

5. **Documentation**: Blackbox AI can help generate API documentation alongside the code, making it easier for frontend developers to integrate.

Educational Value

This approach provides significant educational benefits for your academic project:

1. Students gain exposure to full-stack development concepts
2. Team members learn about API design and database modeling
3. The project demonstrates real-world deployment and integration strategies
4. The implementation process teaches practical GitHub workflow and collaboration.